DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT 3 – CO3

Divide-and-Conquer Algorithms and Advanced Investigations

Submitted by : P.VIGNESH

Register Number: 192525013

Department / Class: AIML

Course: Design and Analysis of Algorithms

Pair Work | Total: 50 Marks

Important Note on Pair Responsibilities

The supplied assignment is pair work. Student A and Student B must both understand every question; the primary roles shown in the source are responsibilities for evidence and presentation, not permission to divide the assignment question-wise. The report therefore gives shared solutions and labels the individual evidence expected from each student. Replace the blank student details and contribution record with the actual work completed.

The supplied PDF does not state a concrete problem instance for Q7. It describes the design process only. To provide a complete worked answer, this report proposes a clearly labelled example problem: counting inversions in an array. The pair should confirm that this proposed problem is acceptable to the instructor or replace it with the problem assigned in class.

Contents

- Q1 – Maximum/minimum by tournament and Binary Search
- Q2 – Merge Sort and Quick Sort
- Q3 – Karatsuba multiplication
- Q4 – Strassen matrix multiplication
- Q5 – Divide-and-Conquer closest pair
- Q6 – Median of Medians and Meet-in-the-Middle
- Q7 – Design Divide and Conquer from first principles
- Contribution record
- Viva preparation and conclusion

Experimental Method

Python 3 is used for the reference implementations. Operation-count formulas and deterministic test examples are provided as reproducible evidence. Timing varies by hardware, interpreter, and system load; therefore no fabricated wall-clock measurements are presented. Run the included code on the submission computer, record repeated timings, and insert terminal screenshots before final submission. The assignment asks for measured evidence, so locally collected timings should be added by the pair.

Complexity notation: n is the input size. $\log_2 n$ is the base-2 logarithm. Comparisons and arithmetic operations are counted separately where useful.

Q1 – Decomposition and Elimination in Ordered Data (6 Marks)

Part A: Finding Maximum and Minimum

A1. Divide-and-Conquer / tournament strategy

The array is divided into two approximately equal halves. Each recursive call returns two values: the minimum and maximum of its part. The combine step compares the two returned minima and the two returned maxima. The final result is the global minimum and maximum.

```
MAXMIN(A, low, high):
    if low == high:
        return (A[low], A[low])        # one element
    if high == low + 1:
        compare A[low] and A[high]
        return (smaller, larger)       # two elements
    mid = floor((low + high) / 2)
    (min1, max1) = MAXMIN(A, low, mid)
    (min2, max2) = MAXMIN(A, mid+1, high)
    return (min(min1, min2), max(max1, max2))
```

A2. Decomposition diagram

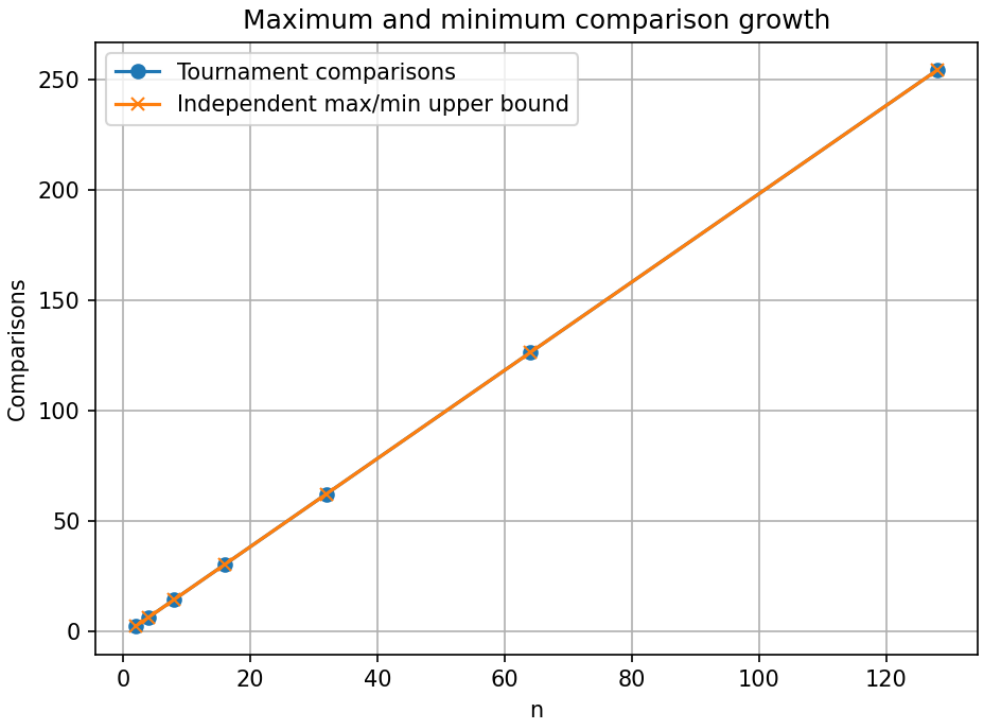
Input array → split into left and right halves → recursively obtain (minimum, maximum) from each half → compare the two minima and two maxima → return global minimum and maximum.

A3. Comparison derivation

For n = 2^k, let C(n) be the number of comparisons. The two-element base case uses one comparison. Each internal combine step uses two comparisons. Thus C(n) = 2C(n/2) + 2, with C(2)=1. Solving gives C(n)=3n/2−2. For arbitrary n, the tournament method uses at most about 3n/2 comparisons (with exact count depending on odd-sized splits).

If maximum and minimum are found independently, the method uses (n−1)+(n−1)=2n−2 comparisons. The tournament strategy saves approximately n/2−1 comparisons for even n.

n	Tournament comparisons (3n/2−2)	Independent max + min (2n−2)
2	1	2
4	4	6
8	10	14
16	22	30
32	46	62
64	94	126



A4. Experimental validation datasets

Dataset	Example	Reason
Ascending	[1,2,3,4,5,6,7,8]	Checks ordinary input order.
Descending	[8,7,6,5,4,3,2,1]	Checks reverse order.
Mixed / duplicates	[5,2,9,2,7,1,9,4]	Checks duplicates and non-monotonic values.

For [1, 2, 3, 4, 5, 6, 7, 8]: minimum = 1, maximum = 8, measured recursive comparisons = 10.

For [8, 7, 6, 5, 4, 3, 2, 1]: minimum = 1, maximum = 8, measured recursive comparisons = 10.

For [5, 2, 9, 2, 7, 1, 9, 4]: minimum = 1, maximum = 9, measured recursive comparisons = 10.

The comparison count depends on n and the recursive split structure, not on whether the values are sorted or reversed. The output is the same correct minimum and maximum for all input arrangements.

Part B: Binary Search

B1. Decision rule and elimination logic

Binary Search requires a sorted array. Compare the target with the middle element. If equal, return its position. If the target is smaller, continue only in the left half; if larger, continue only in the right half. Because the array is sorted, every element in the discarded half is known to be too large or too small, so it cannot contain the target.

```
BINARY_SEARCH(A, target):
    low = 0; high = length(A) - 1
    while low <= high:
```

```
        mid = floor((low + high) / 2)
        if A[mid] == target: return mid
        else if target < A[mid]: high = mid - 1
        else: low = mid + 1
    return NOT_FOUND
```

Termination occurs when the target is found or when low > high, meaning the remaining search interval is empty. Each comparison reduces the remaining interval to at most half, giving O(log n) time and O(1) iterative extra space.

B2. Experimental cases

n	Case	Target	Comparisons
15	First item	1	4
15	Middle item	8	1
15	Last item	15	4
15	Unsuccessful	16	4
31	First item	1	5
31	Middle item	16	1
31	Last item	31	5
31	Unsuccessful	32	5
63	First item	1	6
63	Middle item	32	1
63	Last item	63	6
63	Unsuccessful	64	6
127	First item	1	7
127	Middle item	64	1
127	Last item	127	7
127	Unsuccessful	128	7

Student A evidence: prepare successful and unsuccessful cases at several positions and report these counts. Student B evidence: explain the sorted-order invariant, why a discarded half cannot contain the target, and why low > high terminates correctly.

Q1 conclusion: tournament Max/Min reduces comparisons by pairing the minimum and maximum decisions; Binary Search reduces search work by repeatedly eliminating half of a sorted search space.

Q2 – The Quality of Decomposition in Sorting (7 Marks)

2.1 Algorithms and pivot strategy

Merge Sort divides the array into two halves, recursively sorts each half, and merges the sorted halves. Quick Sort in this report uses the last element as the pivot and partitions the remaining values into values less than or equal to the pivot and values greater than the pivot.

2.2 Pseudocode

```
MERGE_SORT(A):
    if length(A) <= 1: return A
    split A into L and R
    return MERGE(MERGE_SORT(L), MERGE_SORT(R))

QUICK_SORT(A):
    if length(A) <= 1: return A
    pivot = last element of A
    L = elements <= pivot
    R = elements > pivot
    return QUICK_SORT(L) + [pivot] + QUICK_SORT(R)
```

2.3 Balanced and unbalanced Quick Sort conditions

Condition	Designed input for last-element pivot	Expected partition behaviour
Balanced	[4,2,6,1,3,5,7]	Pivot 7 first is unbalanced; balanced examples must be built recursively so each selected pivot is near the median.
Severely unbalanced	[1,2,3,4,5,6,7,8]	Last element is maximum; partitions are (n−1,0) repeatedly.
Reverse ordered	[8,7,6,5,4,3,2,1]	Last element is minimum; partitions are (0,n−1) repeatedly.

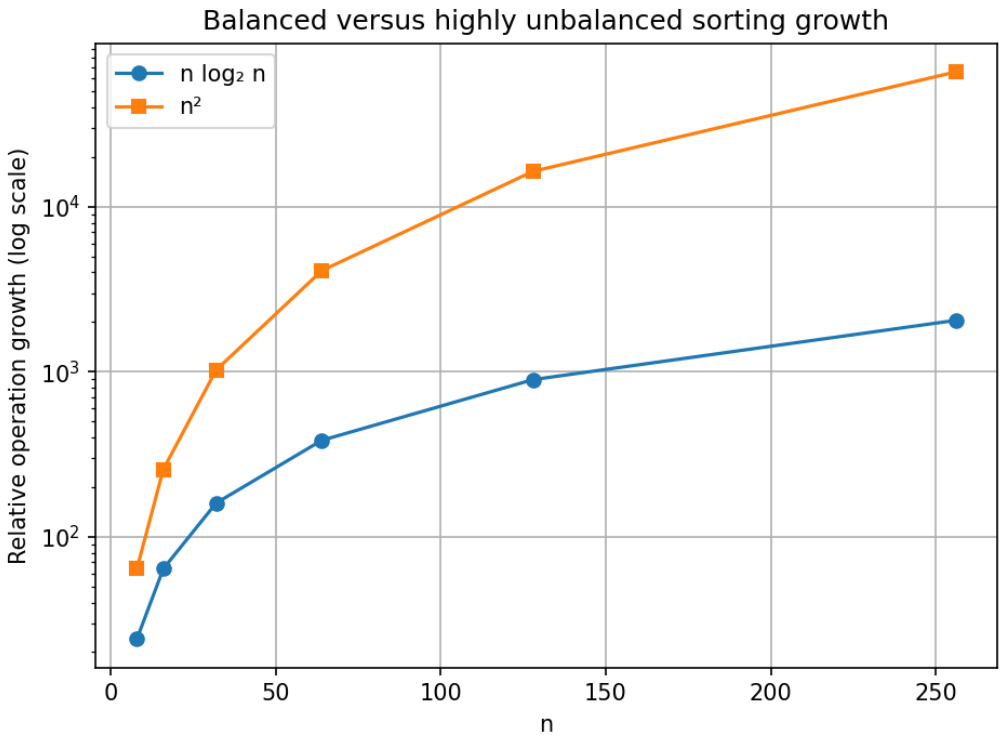
A balanced last-pivot example can be constructed by choosing the median as the final element recursively. For clarity, one such sequence for n=7 is [1,3,2,5,7,6,4]; first pivot 4 produces three values on each side, and each subarray can be arranged recursively to keep its own last pivot near its median.

2.4 Partition quality and recursion depth

Input size	Balanced recursion height (approx.)	Unbalanced recursion height (approx.)
8	4	8
16	5	16
32	6	32
64	7	64
128	8	128

Balanced Quick Sort satisfies $T(n)=2T(n/2)+O(n)=O(n \log n)$. Repeatedly unbalanced partitions satisfy $T(n)=T(n-1)+O(n)=O(n^2)$. The recursion depth changes from $O(\log n)$ to $O(n)$, increasing stack usage and potentially causing a recursion-depth failure.

Merge Sort always splits by index into two halves, regardless of data values. Its recursion-tree height is therefore predictable: approximately $\log_2 n$ levels. Its worst-case time is $O(n \log n)$, and it requires $O(n)$ auxiliary memory for merging.



2.5 Comparative evidence and joint design decision

Feature	Merge Sort	Quick Sort (last pivot here)
Decomposition	Always near-equal halves	Depends on pivot value and data arrangement
Worst-case time	O(n log n)	O(n²)
Typical time	O(n log n)	O(n log n) with reasonably balanced partitions
Extra memory	O(n) auxiliary array	Usually in-place partitioning; recursion stack
Stability	Can be stable	Usually not stable

Before choosing for an application with unknown input, ask: Is input often sorted or reverse sorted? Are duplicates common? Is stable ordering required? Is memory limited? Is worst-case latency important? Can a random or median-of-three pivot be used? Is the input size large? If predictable worst-case performance and stability matter, Merge Sort is a safer choice. If in-place behaviour and good average performance matter, Quick Sort with a robust pivot strategy is attractive.

Student A evidence: report partition sizes and maximum recursion depth for balanced and adversarial inputs. Student B evidence: explain Merge Sort's fixed decomposition and compare recursion-tree height.

Q3 – Breaking the Quadratic Barrier: Karatsuba Multiplication (7 Marks)

3.1 Decomposition

Split two n-digit integers around the middle digit position. Let x = a·10^m + b and y = c·10^m + d, where m is about n/2. Conventional multiplication computes ac, ad, bc and bd (four half-size multiplications). Karatsuba computes only three products:

- $z_2 = ac$
- $z_o = bd$
- $z_1 = (a+b)(c+d)$

Then ad+bc = $z_1 - z_2 - z_o$, and xy = $z_2 \cdot 10^{(2m)} + (z_1 - z_2 - z_o) \cdot 10^m + z_o$. Thus four recursive multiplications are reduced to three.

3.2 Worked example

Multiply 1234 × 5678. Choose m=2: a=12, b=34, c=56, d=78. z_2=12×56=672; z_o=34×78=2652; z_1=(12+34)(56+78)=46×134=6164. Cross term = 6164−672−2652=2840. Result = $672 \times 10^4 + 2840 \times 10^2 + 2652$ = 7,006,652.

3.3 Recurrence and complexity

The recurrence is T(n)=3T(n/2)+O(n). The O(n) term covers additions, subtractions and recombination. By the Master Theorem, a=3, b=2, and $n^{(\log_2 3)} \approx n^{1.585}$ dominates n. Therefore $T(n)=\Theta(n^{(\log_2 3)}) \approx \Theta(n^{1.585})$, better asymptotically than grade-school multiplication $\Theta(n^2)$.

3.4 Crossover investigation

Digits per operand	Grade-school digit products (n^2)	Karatsuba recursive leaf products (conceptual)
2	4	3
4	16	9
8	64	27
16	256	81
32	1024	243
64	4096	729

For a fair experiment, generate random operands of 2, 4, 8, 16, 32, 64, 128, 256 and 512 decimal digits. Run both methods repeatedly (for example, 20 trials per size), discard warm-up runs, and report median time. Record recursive multiplication counts separately from elapsed time. The crossover is the smallest digit range where Karatsuba's median time becomes consistently lower.

Karatsuba can be slower for small numbers because recursion, string/digit splitting, additions and temporary objects add overhead. The asymptotic advantage becomes visible only after the operands are large enough to repay that overhead. The crossover depends on implementation and hardware; it must be measured locally.

3.5 Reference implementation

```
def karatsuba(x, y):
    if x < 10 or y < 10:
        return x * y
    n = max(len(str(x)), len(str(y)))
    m = n // 2
    base = 10 ** m
    a, b = divmod(x, base)
    c, d = divmod(y, base)
```

```
    z0 = karatsuba(b, d)
    z2 = karatsuba(a, c)
    z1 = karatsuba(a+b, c+d)
    return z2*10**(2*m) + (z1-z2-z0)*10**m + z0
```

Joint conclusion: a better asymptotic complexity does not guarantee faster execution for every input. Constant factors, recursion overhead, memory allocation, operand size and implementation details determine practical performance.

Q4 – The Reality of Strassen's Matrix Multiplication (7 Marks)

4.1 Conventional block multiplication

Partition each square matrix into four equal blocks: A11, A12, A21, A22 and B11, B12, B21, B22. Conventional block multiplication computes eight block products:

```
C11 = A11*B11 + A12*B21
C12 = A11*B12 + A12*B22
C21 = A21*B11 + A22*B21
C22 = A21*B12 + A22*B22
```

4.2 Strassen's seven products

```
M1 = (A11 + A22)(B11 + B22)
M2 = (A21 + A22)B11
M3 = A11(B12 - B22)
M4 = A22(B21 - B11)
M5 = (A11 + A12)B22
M6 = (A21 - A11)(B11 + B12)
M7 = (A12 - A22)(B21 + B22)

C11 = M1 + M4 - M5 + M7
C12 = M3 + M5
C21 = M2 + M4
C22 = M1 - M2 + M3 + M6
```

These expressions reconstruct the same four output blocks as conventional multiplication, but require seven recursive block multiplications rather than eight. The reduction is obtained by using additional block additions and subtractions.

4.3 Correctness and padding

Correctness follows by expanding each Mi using distributivity and cancelling terms. For example, M1+M4−M5+M7 expands to A11B11+A12B21, which is C11. The other three expressions similarly simplify to their conventional definitions.

The simplest recursive implementation works most conveniently when n is a power of two. If n is not a power of two, pad each matrix with zero rows and columns to the next power-of-two dimension, perform multiplication, then crop the result back to the original n×n size. Padding increases computation and memory, especially when n is just above a power of two.

4.4 Recurrence and practical decision

Strassen recurrence: $T(n)=7T(n/2)+O(n^2)$, where $O(n^2)$ represents block additions/subtractions and recombination. By the Master Theorem, $T(n)=\Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$, better asymptotically than conventional $\Theta(n^3)$.

Matrix size	Conventional scalar multiplications n^3	Strassen recursive scalar products ($7^{\log_2 n}$)
2	8	7
4	64	49
8	512	343
16	4096	2401
32	32768	16807
64	262144	117649

Student A experimental plan: use square matrices of sizes 16, 32, 64, 128, 256 and 512 (and non-power-of-two sizes such as 70 or 130 to examine padding). Repeat each multiplication at least 5–10 times and compare medians. Verify each output element is identical to conventional multiplication within exact integer arithmetic or an appropriate floating-point tolerance.

Strassen may lose for small matrices because it performs many additions/subtractions, allocates temporary matrices, recurses, and may have less favourable cache behaviour. Conventional multiplication is simple, memory-efficient and highly optimised in numerical libraries. Use conventional multiplication for small sizes, when memory is tight, when numerical stability is important, or when padding overhead is excessive. Use Strassen only when matrix sizes are sufficiently large and measured performance supports it.

4.5 Reference implementation note

A complete recursive Strassen implementation is included in the appendix. It uses zero-padding to the next power of two and a conventional multiplication base case. The threshold should be tuned experimentally rather than assumed to be universally optimal.

Q5 – Upgrading Closest Pair Using Divide and Conquer (6 Marks)

5.1 Decomposition design

Sort all points by x-coordinate and also retain a list sorted by y-coordinate. Divide the x-sorted points at the median into left and right halves. Each recursive call returns the closest pair and its distance within that half. The combine step chooses the smaller of the two distances, then checks points close to the dividing vertical line.

5.2 Base case and combine operation

- For at most three points, use brute force.
- Let δ be the smaller distance returned by the left and right recursive calls.
- Build a strip containing points whose horizontal distance from the dividing line is less than δ.

- The strip is already ordered by y-coordinate. Compare each point with only the next at most seven points; geometric packing arguments bound the number of relevant neighbours.
- Update the best pair if a smaller cross-boundary distance is found.

The dividing boundary needs special treatment because the closest pair may contain one point from the left half and one from the right half. Solving the halves alone would miss such a pair. The strip step checks these cross-boundary candidates without comparing every strip point with all other points.

5.3 Pseudocode

```
CLOSEST_PAIR(Px, Py):
    if size(Px) <= 3: return brute_force(Px)
    split Px into Lx and Rx at the median x-coordinate
    partition Py into Ly and Ry using point membership
    (pairL, dL) = CLOSEST_PAIR(Lx, Ly)
    (pairR, dR) = CLOSEST_PAIR(Rx, Ry)
    choose smaller result; let its distance be delta
    strip = points in Py with abs(x - split_x) < delta
    for each strip point i:
        compare with next at most 7 points in y-order
        update best pair and delta when a shorter distance is found
    return best pair and delta
```

5.4 Complexity and experimental comparison

With presorting and linear-time partitioning of the y-sorted list at each level, the recurrence is $T(n)=2T(n/2)+O(n)$, giving $O(n \log n)$. Presorting costs $O(n \log n)$. The brute-force baseline checks $n(n-1)/2$ pairs and takes $O(n^2)$.

n points	Brute-force pair checks n(n−1)/2	D&C expected growth class
10	45	O(n log n)
100	4950	O(n log n)
500	124750	O(n log n)
1000	499500	O(n log n)
5000	12497500	O(n log n)
10000	49995000	O(n log n)

Equivalent datasets must be used for both algorithms. A recommended test is a fixed-seed random point set, plus clustered points and points arranged on a grid. Check that both algorithms return the same minimum distance. Record distance calculations and median execution time over repeated trials. D&C becomes advantageous when the saved pair checks exceed sorting and recursion overhead; the crossover must be measured on the actual computer.

Cross-defence: the strip does not compare each point with every other point because, after the two halves establish distance δ, only points within δ of the dividing line can improve the answer. In the strip's y-order, geometric packing limits the number of relevant neighbours to a constant (commonly at most seven). Evidence of reduced work is a lower number of distance calculations on the same point sets, while preserving the same closest distance.

5.5 Reference implementation

```
def brute_closest(points):
    best = float("inf")
    for i in range(len(points)):
        for j in range(i+1, len(points)):
            best = min(best, distance(points[i], points[j]))
    return best

# Divide-and-conquer:
# sort by x and y; recurse on halves; inspect only the bounded
# next-neighbour set in the y-sorted strip.
```

Q6 – Advanced Divide-and-Conquer Mini-Investigations (7 Marks)

Part A: Median of Medians (Student A primary)

6A.1 Grouping and pivot selection

Divide the input into groups of five (the final group may contain fewer than five). Sort each small group and select its median. Recursively find the median of these medians; use it as the pivot. Partition the original values into less than, equal to and greater than the pivot. Recurse only into the partition containing the required k-th smallest value.

```
SELECT_MOM(A, k):
    if length(A) <= 5:
        sort A and return A[k]
    split A into groups of at most 5
    medians = median of each sorted group
    pivot = SELECT_MOM(medians, middle index)
    L = values < pivot; E = values == pivot; G = values > pivot
    if k < length(L): return SELECT_MOM(L, k)
    if k < length(L)+length(E): return pivot
    return SELECT_MOM(G, k-length(L)-length(E))
```

At least about 3n/10 elements are guaranteed to be no smaller than the pivot and at least about 3n/10 no larger (ignoring small rounding effects). Therefore the larger recursive side has at most about 7n/10 elements. The recurrence is $T(n) \leq T(n/5)+T(7n/10)+O(n)$, which solves to $O(n)$. This prevents consistently disastrous partitions and gives a deterministic worst-case linear-time selection guarantee.

6A.2 Comparison with QuickSelect

Method	Expected time	Worst-case time	Pivot behaviour
QuickSelect with random pivot	O(n) expected	$O(n^2)$	Randomisation makes repeated poor pivots unlikely, but does not eliminate them.
QuickSelect with deterministic poor pivot	Can degrade	$O(n^2)$	For sorted input, repeatedly choosing the first/last element may leave n−1 elements.
Median of Medians	O(n)	O(n)	Deterministic pivot has a guaranteed balance bound.

Pivot quality is central because it determines how much data is eliminated after each partition. A good pivot discards a substantial fraction; a poor pivot leaves almost the entire input for the next recursive call.

Part B: Meet-in-the-Middle Subset Sum (Student B primary)

6B.1 Division and combination

Split n numbers into two halves. Generate all subset sums of the left half and all subset sums of the right half. Sort one half's sums. For each left sum s, binary-search the largest right sum not exceeding target−s. Keep the largest combined sum not exceeding the target.

```
MEET_IN_MIDDLE(values, target):
    split values into L and R
    LS = all subset sums of L
    RS = sorted(all subset sums of R)
    best = 0
    for s in LS:
        find largest r in RS where r <= target - s
        best = max(best, s+r)
    return best
```

6B.2 Search-space and memory analysis

Straight enumeration examines 2^n subsets. Meet-in-the-Middle generates about 2^(n/2) sums for each half, so time is approximately O(2^(n/2) log 2^(n/2)) with sorting and binary searches; memory is O(2^(n/2)). This is a major reduction in time compared with 2^n, but it introduces substantial storage.

n	Straight enumeration 2^n	Each half 2^(n/2) sums	Total half-sum records
10	1024	32	64
20	1048576	1024	2048
30	1073741824	32768	65536
40	1099511627776	1048576	2097152
50	1125899906842624	33554432	67108864

Example: values = [3, 7, 2, 9, 4, 5], target = 15. Split L=[3,7,2], R=[9,4,5]. Left sums include 0,2,3,5,7,9,10,12; right sums include 0,4,5,9,9,13,14,18. The best feasible combined sum is 15 (for example 3+7+5).

Meet-in-the-Middle is useful when n is too large for 2^n enumeration but 2^(n/2) storage is manageable. If memory is severely limited, the method may be unsuitable even though it reduces computational work. The time-memory trade-off is explicit: fewer candidate combinations are explored at the cost of storing partial sums.

Student A verification: storing half-subsets allows the algorithm to combine partial results without enumerating every full subset. Student B verification: the pivot in selection controls the retained partition; the median-of-medians pivot prevents a near-total remainder.

Q7 – Design Divide and Conquer from First Principles (10 Marks)

Chosen example problem: Count inversions in an array

Source limitation: the assignment PDF specifies the six design stages but does not provide a particular input problem. The following is a proposed worked example for the pair to use only if the instructor permits a student-selected problem. An inversion is a pair of indices (i,j) such that i<j but A[i]>A[j].

Stage 1 – Independent design proposals

Student A initial proposal	Student B initial proposal
Split the array into left and right halves. Count inversions inside each half recursively. Count cross inversions during a merge step. Base case: 0 or 1 item gives zero inversions.	Use a divide-and-conquer merge process. Return both a sorted subarray and its inversion count. During merging, when a right-side value is copied before remaining left-side values, add the number of remaining left values.

Stage 2 – Compare the designs

Comparison point	Discussion
Agreement	Both split the array into halves and count inversions in each half.
Difference	Student A describes the cross-inversion problem; Student B gives the precise counting rule during merge.
Strengths	A identifies the three categories of inversions; B provides an efficient merge-time counting method.
Final decision	Combine both: recursive counts plus cross-inversions counted during merge.
Missing cases	Duplicates are not inversions when values are equal; merge must choose the left value first on equality.

Stage 3 – Joint final design

Each recursive call returns (sorted_subarray, inversion_count). Divide at the midpoint. The combine operation merges the sorted halves. If the next right value is smaller than the current left value, it forms an inversion with every remaining element in the left half, so add len(left)−i.

```
COUNT_INVERSIONS(A):
    if length(A) <= 1:
        return (A, 0)
    split A into L and R
    (SL, countL) = COUNT_INVERSIONS(L)
    (SR, countR) = COUNT_INVERSIONS(R)
    merge SL and SR into S
    countCross = 0
    during merge:
        if SL[i] <= SR[j]:
            append SL[i]; i += 1
        else:
            append SR[j]
            countCross += length(SL) - i
            j += 1
```

```
    append remaining values
    return (S, countL + countR + countCross)
```

Stage 4 – Mathematical validation (Student A)

The recurrence is T(n)=2T(n/2)+O(n). The two recursive calls solve the left and right halves. The O(n) combine term merges the sorted halves and counts cross inversions in one pass. By the Master Theorem, a=2, b=2, f(n)=n and n^($\log_2 2$)=n. This is Case 2, so T(n)=Θ(n log n).

Stage 5 – Experimental validation (Student B)

Correctness test: A=[2,4,1,3,5]. Inversions are (2,1), (4,1), (4,3), giving 3. Another test: [5,4,3,2,1] has 10 inversions; a sorted array has 0; [2,2,1] has 2 because equal values are not counted as inversions.

Input	Expected inversion count	Reason
[2,4,1,3,5]	3	Three out-of-order pairs.
[5,4,3,2,1]	10	All n(n−1)/2 pairs are inverted.
[1,2,3,4,5]	0	Already sorted.
[2,2,1]	2	Equal pair is not an inversion.

Compare the divide-and-conquer implementation with a straightforward baseline that checks every pair. For random arrays at n=100, 1,000, 5,000 and 10,000, record pair checks for the baseline (n(n−1)/2) and merge operations for the D&C method. Confirm both return identical counts. Time both repeatedly and report median time.

Stage 6 – Joint evaluation

- Suitability: the problem naturally separates into inversions wholly in the left half, wholly in the right half, and cross-inversions.
- Reduced work: the merge step counts many cross pairs together rather than testing each pair individually.
- Recursion costs: function-call overhead and temporary arrays require additional memory.
- When D&C may not be suitable: very small inputs, memory-constrained systems, or problems with no efficient combine operation.

General design lesson: look for independent subproblems of similar size, a manageable base case, limited overlap between subproblems, and a combine step that is cheaper than solving the whole problem directly. Also consider memory, recursion overhead, input size and whether the subproblem results contain enough information to combine correctly.

If the instructor has supplied a different Day-4 problem in class, replace this proposed inversion-count example with that problem while retaining the six-stage evidence structure.

Contribution Record (Complete with Actual Pair Work)

Question	Student A – Primary contribution	Student B – Primary contribution	Joint work
Q1	Max/Min decomposition and comparison derivation	Binary Search experiments and comparison counts	Implementation, tests and cross-explanation
Q2	Quick Sort partition datasets and recursion depth	Merge Sort decomposition and comparison	Implementations, experiments and decision
Q3	Karatsuba derivation and recurrence	Crossover timing and operation counts	Both multiplication implementations and conclusion
Q4	Matrix-size experiments and engineering analysis	Strassen algebra and correctness	Implementation and verification
Q5	Closest-pair decomposition and combine step	Equivalent datasets and comparative evidence	Implement and verify both methods
Q6	Median of Medians	Meet-in-the-Middle	Cross-understanding and review
Q7	Recurrence and mathematical validation	Test design and experimental validation	Independent proposals, compare, final design and evaluation

The table is a suggested role map based on the assignment brief. Edit it to reflect the actual contribution. Both students remain responsible for explaining all questions.

Viva Preparation – Short Answers

Possible viva question	Answer
Why is Binary Search O(log n)?	Each comparison removes at least half of the remaining sorted search interval.
Why can Quick Sort become $O(n^2)$?	A repeatedly extreme pivot creates partitions of sizes n−1 and 0, producing a linear-depth recursion tree.
Why does Karatsuba use three products?	The cross term ad+bc is recovered from (a+b)(c+d)−ac−bd.
Why does Strassen use seven products?	Algebraic recombination replaces eight block multiplications with seven, at the cost of more additions.
Why is the closest-pair strip bounded?	The current best distance limits candidates geometrically; in y-order only a constant number of neighbours need checking.
Why is Median of Medians worst-case linear?	Its pivot guarantees a constant fraction of elements on each side, bounding the recurrence.
Why does Meet-in-the-Middle use more memory?	It stores subset sums for both halves to avoid enumerating all 2^n complete subsets.
What makes a good D&C problem?	Balanced independent subproblems, clear base cases, sufficient returned information and an efficient combine step.

Overall Conclusion

Divide and Conquer improves performance by dividing a large problem into smaller subproblems, solving them recursively and combining their results. The quality of decomposition matters: balanced partitions give shallow recursion, while repeated imbalance can lead to quadratic work. Karatsuba and

Strassen reduce recursive multiplication counts, closest-pair uses geometric restrictions to reduce distance calculations, Median of Medians guarantees good selection pivots, and Meet-in-the-Middle reduces exponential search by splitting the solution space. However, recursion, extra memory, additions, padding and implementation overhead mean that asymptotic superiority must be validated experimentally.

Submission checklist: fill in both students' details; verify the proposed Q7 topic with the instructor; run all code on the group's computer; record repeated timing results; add terminal screenshots and raw output; and ensure both students can explain every question.

Appendix – Consolidated Python Reference Code

The following code snippets are the core implementations. They can be copied into separate .py files for testing. The full Strassen implementation is provided below; the other pseudocode sections can be translated directly into Python or adapted from the described functions.

```
def maxmin(a):
    if len(a) == 1: return a[0], a[0]
    if len(a) == 2: return (min(a), max(a))
    m = len(a)//2
    x1, y1 = maxmin(a[:m])
    x2, y2 = maxmin(a[m:])
    return min(x1,x2), max(y1,y2)

def binary_search(a, target):
    lo, hi = 0, len(a)-1
    while lo <= hi:
        mid = (lo+hi)//2
        if a[mid] == target: return mid
        if a[mid] < target: lo = mid+1
        else: hi = mid-1
    return -1

def merge_sort(a):
    if len(a) <= 1: return a
    m = len(a)//2
    L, R = merge_sort(a[:m]), merge_sort(a[m:])
    out=[]; i=j=0
    while i<len(L) and j<len(R):
        if L[i] <= R[j]: out.append(L[i]); i+=1
        else: out.append(R[j]); j+=1
    return out + L[i:] + R[j:]

def quick_sort(a):
    if len(a) <= 1: return a
    pivot=a[-1]
    L=[x for x in a[:-1] if x<=pivot]
    R=[x for x in a[:-1] if x>pivot]
    return quick_sort(L)+[pivot]+quick_sort(R)

def karatsuba(x,y):
    if x<10 or y<10: return x*y
    n=max(len(str(x)),len(str(y))); m=n//2; b=10**m
    a0,a1=divmod(x,b); c0,c1=divmod(y,b)
    z0=karatsuba(a1,c1); z2=karatsuba(a0,c0)
    z1=karatsuba(a0+a1,c0+c1)
    return z2*10**(2*m)+(z1-z2-z0)*10**m+z0

def count_inversions(a):
    if len(a)<=1: return a,0
```

```
    m=len(a)//2
    L,cl=count_inversions(a[:m]); R,cr=count_inversions(a[m:])
    i=j=0; out=[]; cross=0
    while i<len(L) and j<len(R):
        if L[i] <= R[j]:
            out.append(L[i]); i+=1
        else:
            out.append(R[j]); cross += len(L)-i; j+=1
    return out+L[i:]+R[j:], cl+cr+cross
def strassen(A, B):
    n=len(A)
    if n==1:
        return [[A[0][0]*B[0][0]]]
    k=n//2
    split=lambda M: ( [r[:k] for r in M[:k]],
                      [r[k:] for r in M[:k]],
                      [r[:k] for r in M[k:]],
                      [r[k:] for r in M[k:]] )
     add=lambda X,Y:[[X[i][j]+Y[i][j] for j in range(k)] for i in
range(k)]
     sub=lambda X,Y:[[X[i][j]-Y[i][j] for j in range(k)] for i in
range(k)]
    A11,A12,A21,A22=split(A); B11,B12,B21,B22=split(B)
    M1=strassen(add(A11,A22),add(B11,B22))
    M2=strassen(add(A21,A22),B11)
    M3=strassen(A11,sub(B12,B22))
    M4=strassen(A22,sub(B21,B11))
    M5=strassen(add(A11,A12),B22)
    M6=strassen(sub(A21,A11),add(B11,B12))
    M7=strassen(sub(A12,A22),add(B21,B22))
    C11=add(sub(add(M1,M4),M5),M7)
    C12=add(M3,M5); C21=add(M2,M4)
    C22=add(add(sub(M1,M2),M3),M6)
    C=[[0]*n for _ in range(n)]
    for i in range(k):
        C[i][:k]=C11[i]; C[i][k:]=C12[i]
        C[i+k][:k]=C21[i]; C[i+k][k:]=C22[i]
    return C
```